Projeto_ProCafeeiro

Gabriel G Ruas

Apresentação

- O que e’ o Projeto Procafeeiro
- Api
- App
- Hardware

Gabrel G Ruas

Apresentação

- O que e’ o Projeto Procafeeiro

O projeto vai monitorar as Panha
O projeto vai fazer a parte finaceira da lavoura
O projeto vai monitorar a frota

Gabrel G Ruas

API

- Dominio
- Repositorio
- Aplicacao
- Aplesentacao
- Serviços

Gabrel G Ruas

APP

- Pagina
- Panha.js
- Panha.modules.css
- PanhaAPI

Gabrel G Ruas

Harware

- Esp32
- Planilha google
- C++
- ModuloGPS
- Googlemaps

Gabrel G Ruas

CÓDIGO ESP32 – GPS + SD + WiFi + NTP + HTTP

Cada parte do código abaixo corresponde a uma função/módulo do programa.

Codigo C++

Gabrel G Ruas

1 BIBLIOTECAS

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <TinyGPS++.h>
#include <HardwareSerial.h>
#include <SD_MMC.h>
#include <SD.h>
#include <SPI.h>
#include <NTPClient.h>
#include <WiFiUdp.h>
#include <ArduinoJson.h>
```

2 DEFINIÇÕES E ESTRUTURAS

```
const char* filename = "/config.txt";

struct WiFiCredentials {
  String ssid;
  String password;
};

std::vector<WiFiCredentials> wifiCredentialsList;
String serverUrl;

#define GPS_RX_PIN 12  // Pino RX do GPS
#define GPS_TX_PIN 13  // Pino TX do GPS

TinyGPSPlus gps;
HardwareSerial gpsSerial(1);

const long utcOffsetInSeconds = -3 * 3600; // GMT-3

WiFiUDP ntpUDP;
NTPClient timeClient(ntpUDP, "pool.ntp.org",
 utcOffsetInSeconds);

int registro = 1; // Número inicial do registro
```

3 FUNÇÃO: printDateAndTime

```
String printDateAndTime(unsigned long epochTime) {
  // Converte timestamp em string YYYY-MM-DDTHH:MM:SS
  const unsigned long SECONDS_PER_DAY = 86400;
  const unsigned long SECONDS_PER_HOUR  = 3600;
  const unsigned long SECONDS_PER_MINUTE = 60;

  int days = epochTime / SECONDS_PER_DAY;
  int hours = (epochTime % SECONDS_PER_DAY) / SECONDS_PER_HOUR;
  int minutes = (epochTime % SECONDS_PER_HOUR)  / SECONDS_PER_MINUTE;
  int seconds = epochTime % SECONDS_PER_MINUTE;

  // Cálculo de ano, mês, dia (sem biblioteca de data)
  int year = 1970;
  while (days >= 365) {
    if (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
      if (days >= 366) days -= 366; else break;
    } else {
      days -= 365;
    }
    year++;
  }

  int month = 0, day = 0;
  static const int monthDays[12] = {31,28,31,30,31,30,31,31,30,31,30,31};
  for (month = 0; month < 12; month++) {
    int daysInMonth = monthDays[month];
    if (month == 1 && ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)))
      daysInMonth = 29;
    if (days < daysInMonth) break;
    days -= daysInMonth;
  }
  day = days + 1;

  String formattedDateTime = String(year) + "-" + (month + 1 < 10 ? "0" : "") +
     String(month + 1) + "-" + (day < 10 ? "0" : "") + String(day) + "T" +
     (hours < 10 ? "0" : "") + String(hours) + ":" +
     (minutes < 10 ? "0" : "") + String(minutes) + ":" +
     (seconds < 10 ? "0" : "") + String(seconds);
  return formattedDateTime;
}
```

4 FUNÇÃO: setup()

```
void setup() {
  Serial.begin(115200);
  delay(1000);

  // Inicializar cartão SD
  if (!SD_MMC.begin("/sdcard", true)) {
    Serial.println("Falha ao inicializar o cartão SD!");
    return;
  }
  Serial.println("Cartão SD inicializado com sucesso!");

  // Abrir arquivo de configuração
  File file = SD_MMC.open(filename);
  if (!file) {
    Serial.println("Falha ao abrir o arquivo");
    return;
  }

  // Ler dados do arquivo
  while (file.available()) {
    String line = file.readStringUntil('\n');
    line.trim();
    if (line.startsWith("ssid:")) {
      int commaIndex = line.indexOf(',');
      if (commaIndex > 5) {
        WiFiCredentials creds;
        creds.ssid = line.substring(5, commaIndex);
        creds.password = line.substring(commaIndex + 10);
        wifiCredentialsList.push_back(creds);
      }
    } else if (line.startsWith("serverUrl:")) {
      serverUrl = line.substring(10);
    }
  }
  file.close();

  // Conectar ao WiFi
  Serial.println("Conectando às redes Wi-Fi...");
  for (const auto& creds : wifiCredentialsList) {
   Serial.print("Tentando conectar-se a ");
   Serial.println(creds.ssid);
   WiFi.begin(creds.ssid.c_str(), creds.password.c_str());
    delay(5000);
    if (WiFi.status() == WL_CONNECTED) {
      Serial.println("\nConectado à rede Wi-Fi");
      break;
    } else {
      Serial.println("\nFalha na conexão à rede Wi-Fi");
    }
  }
  // Inicializar GPS (Serial1)
  gpsSerial.begin(9600, SERIAL_8N1, GPS_RX_PIN, GPS_TX_PIN);

  // Iniciar cliente NTP
  timeClient.begin();
}
```

5 FUNÇÃO: loop()

```
void loop() {
  if (Serial.available() > 0) {
    String input = Serial.readStringUntil('\n');
    if (input == "gps") {
      getGPSData();
    } else if (input == "sdget") {
      readSDData();
    } else if (input == "sddel") {
      clearSDFile();
    } else if (input == "sdsend") {
      verificarEnvioCartaoSD();
    } else if (input == "sdreg") {
      fetchData();
    } else if (input == "sdenvi") {
      verificarEnvioCartaoSD();
    } else if (input == "sdadd") {
      saveToSD("-21.258810", "-46.141857", "10:26:33");
    } else if (input == "sdsendd") {
      int enviado = sendToGoogleSheets("-21.258810",
                                       "-46.1418S7", "10:26:33");
      Serial.println("Resultado do envio");
      Serial.println(enviado);
    }
  }

  // Atualizar hora NTP
  timeClient.update();
  Serial.print("Data e Hora: ");
  Serial.println(printDateAndTime(timeClient.getEpochTime()));

  // Ler dados do GPS por 1 segundo
  smartdelay_gps(1000);

  if (gps.location.isValid()) {
    double lat = gps.location.lat();
    double lng = gps.location.lng();
    String dataHora = printDateAndTime(timeClient.getEpochTime());
    Serial.print("LAT: "); Serial.println(lat, 6);
    Serial.print("LNG: "); Serial.println(lng, 6);
    Serial.print("Data/Hora: "); Serial.println(dataHora);
    saveToSD(String(lat, 6), String(lng, 6), dataHora);
  } else {
    Serial.println("Aguardando dados válidos do GPS...");
  }
   delay(1000);
}
```

6 FUNÇÃO: sendToGoogleSheets

```
int sendToGoogleSheets(String lat, String lng, String dataHora) {
  if (WiFi.status() != WL_CONNECTED) return 0;

  HTTPClient http;
  String url = serverUrl + "?latitude=" + lat +
               "&longitude=" + lng +
               "&data=" + dataHora;

  http.begin(url);
  int httpCode = http.GET();
  int resultado = (httpCode == 200) ? 1 : 0;
  http.end();
  return resultado;
}
```

7 FUNÇÃO: saveToSD

```
void saveToSD(String lat, String lng, String dataHora) {
  File file = SD_MMC.open("/data.txt", FILE_APPEND);
  if (!file) {
    Serial.println("Erro ao abrir /data.txt");
    return;
  }

  file.printf("Registro: %d\n", registro++);
  file.printf("Latitude: %s\n", lat.c_str());
  file.printf("Longitude: %s\n", lng.c_str());
  file.printf("Data: %s\n", dataHora.c_str());
  file.println("Enviado: nao");

  file.println("-----------------------------------");
  file.close();

}
```

8 FUNÇÃO: getGPSData

```
void getGPSData() {
  smartdelay_gps(1000);
  if (gps.location.isValid()) {
    Serial.print("LAT: ");
    Serial.println(gps.location.lat(), 6);
    Serial.print("LNG: ");
    Serial.println(gps.location.lng(), 6);
  } else {
    Serial.println("GPS sem posição válida!");
  }
}
```

9 FUNÇÃO: readSDData

```
void readSDData() {
  File file = SD_MMC.open("/data.txt");
  if (!file) {
    Serial.println("Erro ao abrir /data.txt");
    return;
  }
  while (file.available()) {
    Serial.write(file.read());
  }
  file.close();
}
```

10 FUNÇÃO: clearSDFile

```
void clearSDFile() {
  File file = SD_MMC.open("/data.txt", FILE_WRITE);
  if (!file) {
    Serial.println("Erro ao limpar /data.txt");
    return;
  }
  file.print("");  // Limpa o arquivo
  file.close();
  Serial.println("Arquivo /data.txt limpo com sucesso!");
}
```

11 FUNÇÃO: verificarEnvioCartaoSD

```
void verificarEnvioCartaoSD() {
  // Lê /data.txt, envia registros com "Enviado: nao"
  // Usa sendToGoogleSheets()
  // Atualiza para "Enviado: sim" se sucesso
  // Regrava o arquivo
}
```

12 FUNÇÃO: fetchData

```
void fetchData() {
  // Faz GET em serverUrl
  // Lê JSON e imprime o campo "registro"
}
```

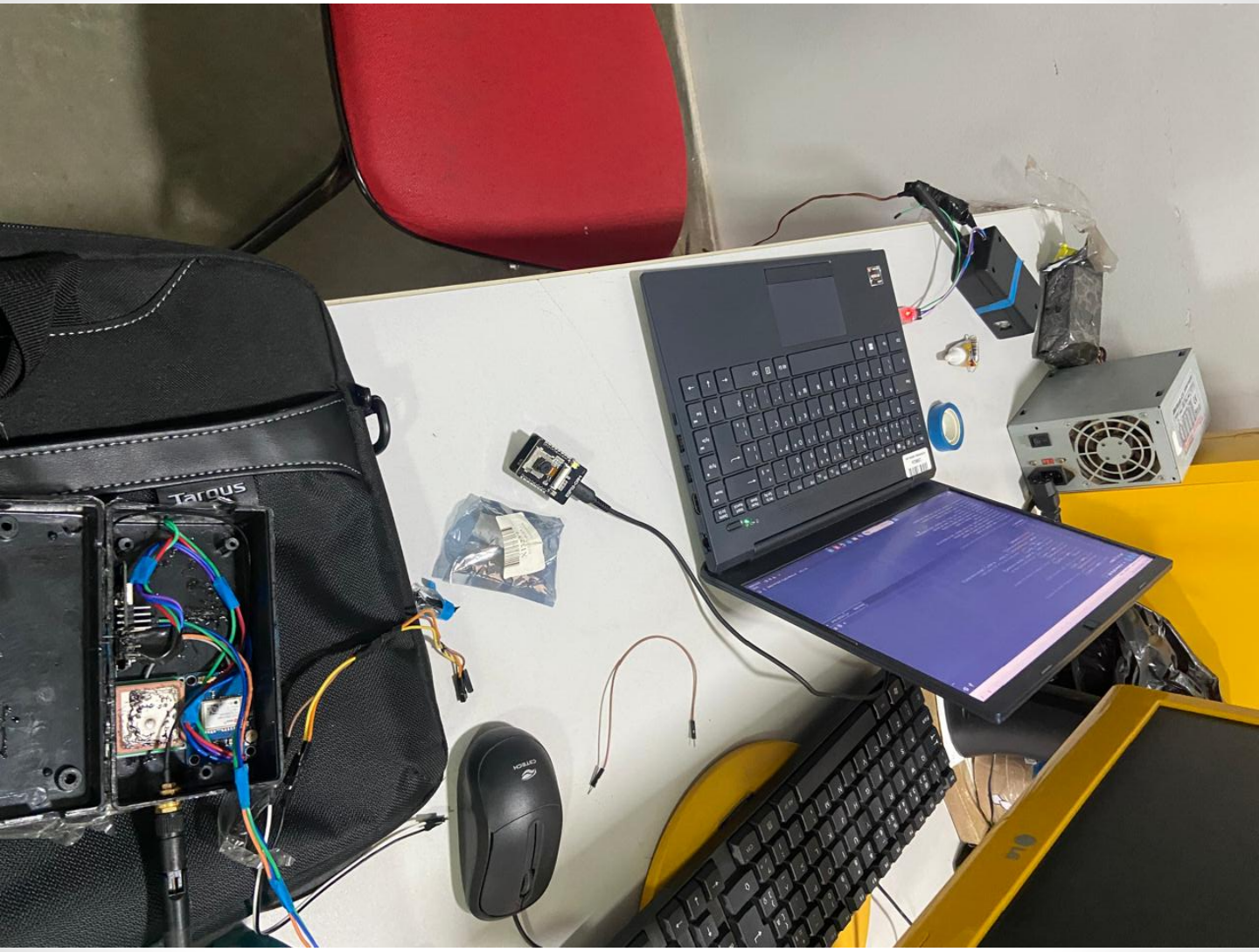
13 FUNÇÃO: smartdelay_gps

```
void smartdelay_gps(unsigned long ms) {
  unsigned long start = millis();
  do {
    while (gpsSerial.available() > 0) {   gps.encode(c);
          c = gpsSerial.read();
    }
  } while (millis() - start < ms); }
```

LEGENDA

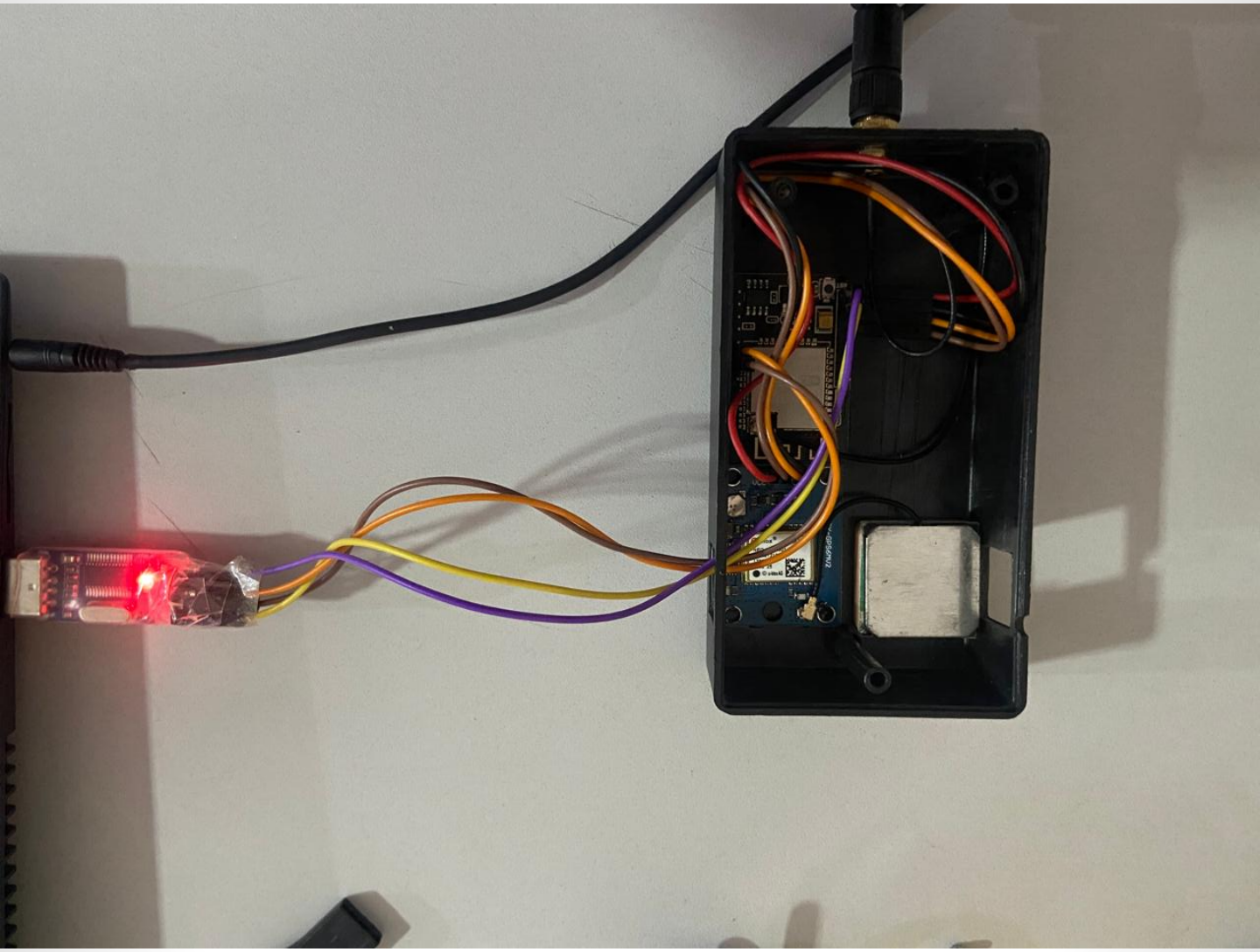
- Azul: Comunicação / Tempo
- Verde: Arquivos / SD
- Roxo: Funções de leitura
- Laranja: Envio / Rede
- Marrom: Utilitários / Controle

Hardware GPS ESP32Can



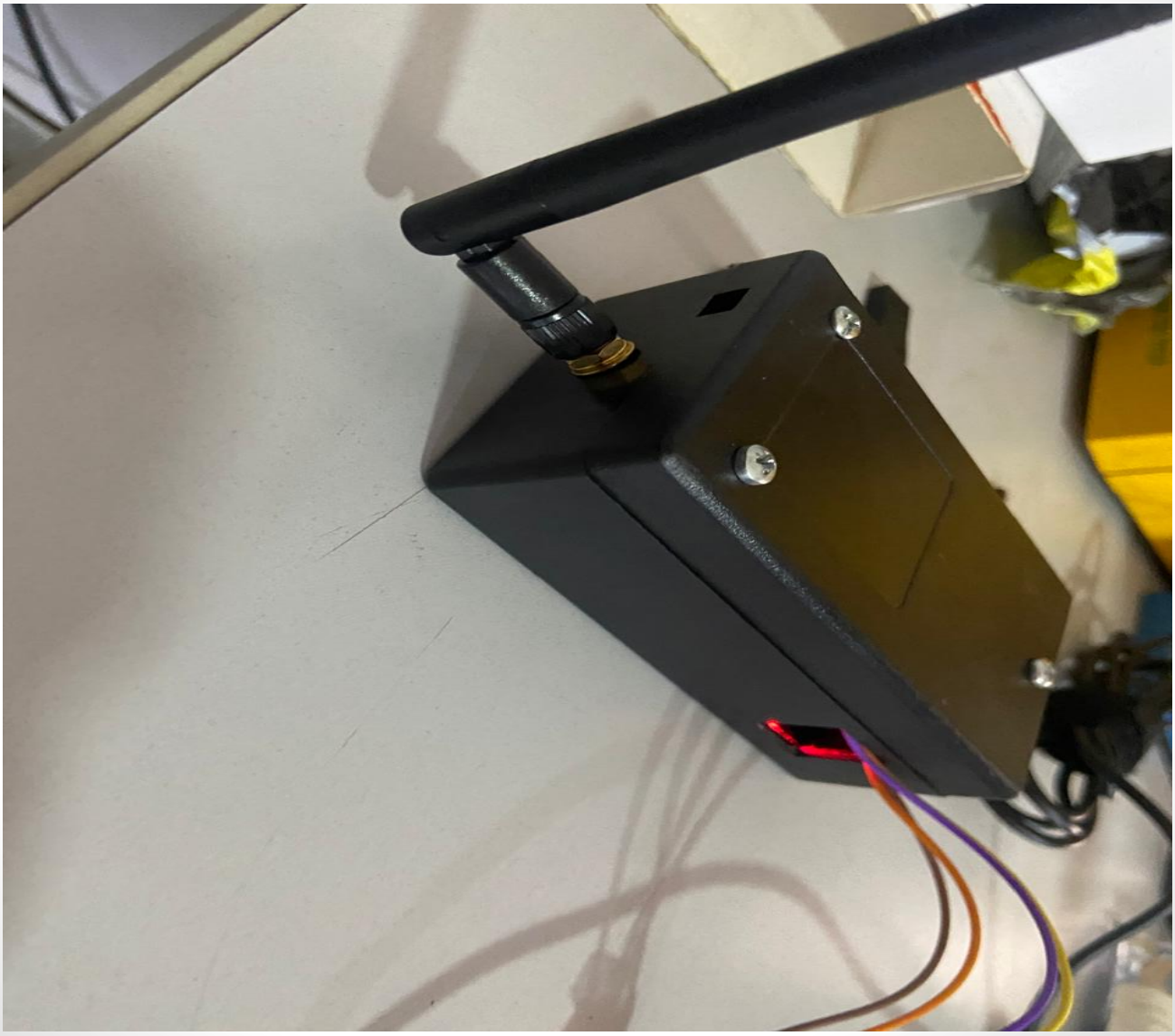
Gabrel G Ruas

Hardware GPS ESP32Can



Gabrel G Ruas

Hardware GPS ESP32Can



Gabrel G Ruas